# Python Modules & Packages — Teaching Module

**Duration:** ~90 minutes | **Level:** Beginner to Intermediate | **Format:** Concept + Live Coding

---

## Learning Objectives

By the end of this module, learners will be able to: 1. Explain what a module and a package are, and why Python uses them 2. Create, import, and reuse their own modules 3. Use all major import styles (`import`, `from...import`, aliasing, wildcard) 4. Build a multi-file package with `__init__.py` 5. Understand `__name__ == "__main__"` and when code runs vs. doesn't 6. Explore standard library modules and third-party packages via `pip` 7. Understand Python's module search path (`sys.path`) and how imports are resolved

---

## 1. Why Modules and Packages? (10 min — Concept)

Imagine you're building a **Retail Analytics System** for a company with stores in Indore, Pune, and Jaipur. As the codebase grows, you'll have:

- Code to clean sales data
- Code to calculate discounts and taxes
- Code to generate reports
- Code to connect to a database

If all of this lives in **one giant `.py` file**, it becomes unreadable, hard to test, and impossible to reuse. This is where **modules** and **packages** come in.

| Term | What it is | Real-world analogy |
|---|---|---|
| **Module** | A single `.py` file containing Python code (functions, classes, variables) | One chapter of a book |
| **Package** | A folder containing multiple related modules + an `__init__.py` file | The whole book (collection of chapters), organized in a table of contents |
| **Library** | A collection of packages bundled for distribution (e.g., `pandas`) | A whole bookshelf on one subject |

**Key benefits:** - **Reusability** — write once, import anywhere - **Namespacing** — avoid naming conflicts (`sales.report()` vs `inventory.report()`) - **Organization** — logically group related functionality - **Maintainability** — easier to debug and update small files than one 5,000-line script

---

## 2. Creating and Using Your Own Module (15 min — Live Coding)

A module is simply **any `.py` file**. Let's create one for our retail system.

### Step 1: Create the module

**File:** `discount.py`

```
# discount.py
# A simple module to calculate discounts for a retail store

STORE_NAME = "Indore Retail Hub"

def apply_discount(price, discount_percent):
    """Apply a percentage discount to a price."""
    discount_amount = price * (discount_percent / 100)
    final_price = price - discount_amount
    return round(final_price, 2)

def festive_discount(price):
    """Fixed 20% festive season discount."""
    return apply_discount(price, 20)

class Customer:
    def __init__(self, name, loyalty_points=0):
        self.name = name
        self.loyalty_points = loyalty_points

    def redeem_points(self, points_needed):
        if self.loyalty_points >= points_needed:
            self.loyalty_points -= points_needed
            return True
        return False
```

### Step 2: Use the module in another file

**File:** `main.py` (in the same folder)

```
# main.py
import discount

price = 1500
final_price = discount.apply_discount(price, 10)
print(f"Store: {discount.STORE_NAME}")
print(f"Final price after 10% discount: ₹{final_price}")

festive_price = discount.festive_discount(price)
print(f"Festive price: ₹{festive_price}")

cust = discount.Customer("Ravi Sharma", loyalty_points=500)
print(cust.redeem_points(200))  # True
print(cust.loyalty_points)      # 300
```

**Output:**

```
Store: Indore Retail Hub
Final price after 10% discount: ₹1350.0
Festive price: ₹1200.0
True
300
```

> **Teaching note:** Emphasize that `discount.py` and `main.py` must be in the **same directory** (or the module's location must be on `sys.path` — covered in Section 6).

---

## 3. Import Styles (15 min — Concept + Coding)

There are four main ways to import. Let's use `discount.py` from above to demonstrate each.

### 3.1 `import module_name`

```
import discount
print(discount.apply_discount(1000, 15))
```

Access everything with the `module.` prefix. Safest — avoids name clashes.

### 3.2 `from module_name import specific_item`

```
from discount import apply_discount, Customer

print(apply_discount(1000, 15))
cust = Customer("Neha Verma")
```

Import only what you need, use it directly without the prefix.

### 3.3 `import module_name as alias`

```
import discount as disc

print(disc.festive_discount(2000))
```

Common convention for long module names (e.g., `import pandas as pd`, `import numpy as np`).

### 3.4 `from module_name import *` (wildcard — use with caution!)

```
from discount import *

print(apply_discount(1000, 15))
print(STORE_NAME)
```

⚠ **Teaching point:** This imports *everything* public from the module into your namespace, which can silently overwrite existing variable/function names. Explain this is generally **discouraged** in production code — best used only in quick scripts or interactive sessions.

---

## 4. `__name__ == "__main__"` (10 min — Concept + Coding)

Every Python file has a built-in variable `__name__`. When a file is **run directly**, `__name__` is set to `"__main__"`. When it's **imported**, `__name__` is set to the module's filename.

**File:** `discount.py` (add this at the bottom)

```
if __name__ == "__main__":
    # This block runs ONLY when discount.py is executed directly,
    # NOT when it's imported into another file
    print("Running discount.py directly — testing the module...")
    print(apply_discount(500, 10))
```

**Try it two ways:**

```
python discount.py
```

Output:

```
Running discount.py directly — testing the module...
450.0
```

```
python main.py
```

Output: (the test block does **not** run — only `main.py`'s own code executes)

> **Why this matters:** This pattern lets you write test/demo code inside a module without it accidentally running every time someone else imports your module.

---

## 5. Building a Package (20 min — Live Coding, the core exercise)

A **package** is a folder of related modules. Let's build a `retail_toolkit` package for our system with modules for pricing, inventory, and reporting.

### Folder structure

```
retail_project/
│
├── main.py
└── retail_toolkit/
    ├── __init__.py
    ├── pricing.py
    ├── inventory.py
    └── reports.py
```

**`retail_toolkit/pricing.py`**

```
def apply_tax(price, gst_percent=18):
    return round(price + (price * gst_percent / 100), 2)

def apply_discount(price, percent):
    return round(price - (price * percent / 100), 2)
```

**`retail_toolkit/inventory.py`**

```
stock = {
    "Indore": 120,
    "Pune": 85,
    "Jaipur": 60
}

def check_stock(city):
    return stock.get(city, "City not found")

def restock(city, quantity):
    if city in stock:
        stock[city] += quantity
    return stock
```

**`retail_toolkit/reports.py`**

```
from .pricing import apply_tax  # relative import within the package

def generate_invoice(item, price, city):
    final_price = apply_tax(price)
    return f"Invoice | {item} | City: {city} | Total (incl. GST): ₹
{final_price}"
```

**`retail_toolkit/__init__.py`**

```
# __init__.py tells Python "this folder is a package"
# It can be empty, OR used to control what gets exposed on import
```

```
from .pricing import apply_tax, apply_discount
from .inventory import check_stock, restock
from .reports import generate_invoice

print("retail_toolkit package initialized")
```

### Using the package from `main.py`

```
from retail_toolkit import apply_tax, check_stock, generate_invoice

price_with_tax = apply_tax(1200)
print(f"Price with GST: ₹{price_with_tax}")

print(check_stock("Pune"))

invoice = generate_invoice("Wireless Mouse", 799, "Indore")
print(invoice)
```

**Output:**

```
retail_toolkit package initialized
Price with GST: ₹1416.0
85
Invoice | Wireless Mouse | City: Indore | Total (incl. GST): ₹942.82
```

> **Teaching points to call out:** - `__init__.py` is what makes Python treat a folder as a **package** (required in older Python; optional-but-recommended for clarity in Python 3.3+ with "namespace packages") - The `.` in `from .pricing import apply_tax` is a **relative import** — it means "from a module inside this same package" - `__init__.py` can act as the package's "front door," exposing a clean public API so users don't need to know internal file structure

### Nested (sub-)packages

```
retail_toolkit/
├── __init__.py
├── pricing.py
├── inventory.py
├── reports.py
└── analytics/
    ├── __init__.py
    └── trends.py
```

```
# Importing from a sub-package
from retail_toolkit.analytics.trends import monthly_growth
```

---

## 6. How Python Finds Modules — `sys.path` (10 min — Concept + Coding)

When you `import` something, Python searches locations in this order: 1. The **current directory** (where the script is run from) 2. Directories listed in the `PYTHONPATH` environment variable 3. Standard library install locations 4. Site-packages (where `pip`-installed packages live)

```
import sys
print(sys.path)  # shows the full search path as a list of strings

# You can temporarily add a folder to the search path:
sys.path.append("/home/prachi/shared_modules")
```

> **Common error to demo deliberately:** `ModuleNotFoundError: No module named 'discount'` — happens when running a script from a different directory than the module. Great moment to show `sys.path` and explain

> *why* the error occurred.

---

## 7. Standard Library & Third-Party Packages (10 min — Concept + Coding)

### Standard library (built into Python — no install needed)

```
import math
import random
import datetime
import os

print(math.sqrt(144))                    # 12.0
print(random.choice(["Indore", "Pune", "Jaipur"]))
print(datetime.date.today())
print(os.getcwd())                       # current working directory
```

### Third-party packages (installed via `pip`)

```
pip install pandas
```

```
import pandas as pd

df = pd.DataFrame({
    "City": ["Indore", "Pune", "Jaipur"],
    "Sales": [45000, 62000, 38000]
})
print(df)
```

| Type | Install needed? | Example |
|---|---|---|
| Built-in module | No | `math`, `os`, `datetime`, `random` |
| Third-party package | Yes (`pip install`) | `pandas`, `numpy`, `requests` |
| Your own module/package | No (just needs to be on the path) | `discount.py`, `retail_toolkit/` |

---

## 8. Hands-On Exercise (for learners)

**Task:** Build a package called `hr_toolkit` for an HR system with: - `hr_toolkit/attendance.py` — function `mark_present(employee_id)` that appends to a list - `hr_toolkit/payroll.py` — function `calculate_salary(base, days_present)` - `hr_toolkit/__init__.py` — expose both functions at the package level - A `main.py` that imports and uses both

**Bonus challenge:** Add a `if __name__ == "__main__":` test block inside `payroll.py` that prints a sample salary calculation when run directly.

---

## 9. Quick Recap / Cheat Sheet

```
# Import styles
import module_name
from module_name import thing
import module_name as alias
from module_name import *          # avoid in production
```

```
# Package essentials
retail_toolkit/
    __init__.py                    # makes it a package
    module1.py
    module2.py

# Relative import inside a package
from .module1 import function_name

# Run-only-if-main pattern
if __name__ == "__main__":
    # test/demo code here
    pass

# Inspecting the search path
import sys
print(sys.path)
```

---

## 10. Common Mistakes to Flag in Class

1. **Circular imports** — Module A imports Module B, which imports Module A → `ImportError`
2. Forgetting `__init__.py` in older Python versions or when explicit package structure is wanted
3. Naming your own file the same as a standard library module (e.g., `random.py`) — shadows the real one and breaks things
4. Using `from module import *` and overwriting existing names unknowingly
5. Running a script from the wrong directory, causing `ModuleNotFoundError`